

Native installers

Adobe Air 2.0 can be used to create native installers for multiple operating systems

PixelBender

You can develop custom custom filters for Adobe Flash that can be used to process image and other kinds of data

Developer corner

The first option is labelled “Package”. When you create a new libraries of code, it is a good idea to organise them so that they can be reused later on without problems. If you are creating a new Class for retrieving stocks information from Yahoo! called “YahooStocks” for a project called “Stock Manager”, you might want to have a package such as `com.stockmanager`. Although it is recommended that you always place your classes inside a package, we will do without one here since it is a simple example.

The second option is for a name for your class. We are giving our class a name of “FeedTab”. The convention is to have a class name where each word begins with a capital such as `MultiTouchManager` or `ArrayCollection`.

Superclass is used if you are extending an already existing class. Those familiar with object-oriented programming will understand what this means. Those unfamiliar, should look up inheritance in object-oriented programming. You could use this feature for example to create your custom version of the “Video” class called “SubtitledVideo” to provide support for subtitles.

Interfaces are a way of making different classes compatible with each other. For example, while the `ArrayCollection`, and `XMLListCollection` classes are very different — one stores data as an unstructured array, and the other as structured XML — both can be used as a source for the list. This is because both implement the `IList` interface. For them to be part of a list they both need to provide a few features, such as moving forward and backward in a list and getting the length of the list. If you create your own custom class which implements `IList`, it too will be usable.

This is the entirety of our “FeedTab” class:

```
package
{
    import mx.collections.
XMLListCollection;
    public class FeedTab
    {
```

```
        public var title:String
        public var
data:XMLListCollection;
        public function
FeedTab(title:String,
        data:XMLListCollection){
            this.title = title;
            this.data = data;
        }
    }
}
```

All it is doing is storing the data needed by our tab, which is: a title for the feed, and the feed data itself. The feed data itself contains a title, however this makes things a little bit simpler. The `FeedTab` function which serves as a constructor for this Class takes two the two pieces of data this class needs and stores them in the public variables which make up this class.

Finally we will modify our main code. First of all remove the variable `feedData`, we will no longer need it as we handle multiple feeds. Secondly, we will add a new variable, and array to store all our feeds’ data.

```
[Bindable]
private var
feeds:ArrayCollection = new
ArrayCollection();
```

Now in our `onFeedLoaded` function, we will need to add this feed to our list of feeds instead.

```
private function
onFeedLoaded(event:Event):void {
    var rssFeed:XML =
XML((event.target as URLLoader).
data);
```

```
        var
feedData:XMLListCollection
            = new
XMLListCollection(rssFeed.
channel.item);
        var feedTab:FeedTab
            = new
FeedTab(rssFeed.channel.titles,
feedData);
        feeds.addItem(feedTab);
        feedTabs.selectedItem =
feed;
```

The first line remains unchanged, and in the second line we are temporarily storing the `XMLListCollection` made up of all the items in the RSS feed, in a variable called `feedData`. Then we go on to create a new `FeedTab` item (which is the class we just created), by giving it the title of the feed (which can be found under `channel.item` of the RSS feed), and the `feedData`. We then push this `FeedTab` item into our list of feeds, and finally make the newly added `FeedTab` the new selected tab.

We should now go ahead and add “feeds” as the dataProvider for the `ButtonBar`, and set the `ButtonBar`’s `labelField` attribute to “title” since that is the name of the field where we are storing the title of the feed that is to be used as a label for the button.

Finally, we need to handle removing tabs. This is easily done by adding a click handler for the close button with the following simple line of code:

```
feeds.removeItemAt(feedTabs.
selectedIndex);
```

Here `feedTabs` is the id of the `ButtonBar` that lists the feed items. We are removing the item at the `selectedIndex` of the `ButtonBar` based tab bar, from the feeds array. This is only working as expected as the order of items in the feeds array and the tab bar is the same.

Believe it or not, at this point we have a functional multi-tab feed browser! Go ahead and test your code.

For an expanded version of this tutorial and the full code of this application you can check out this article online at http://www.thinkdigit.com/d/fb4_tut1_5.



You now have an RSS browser of your own